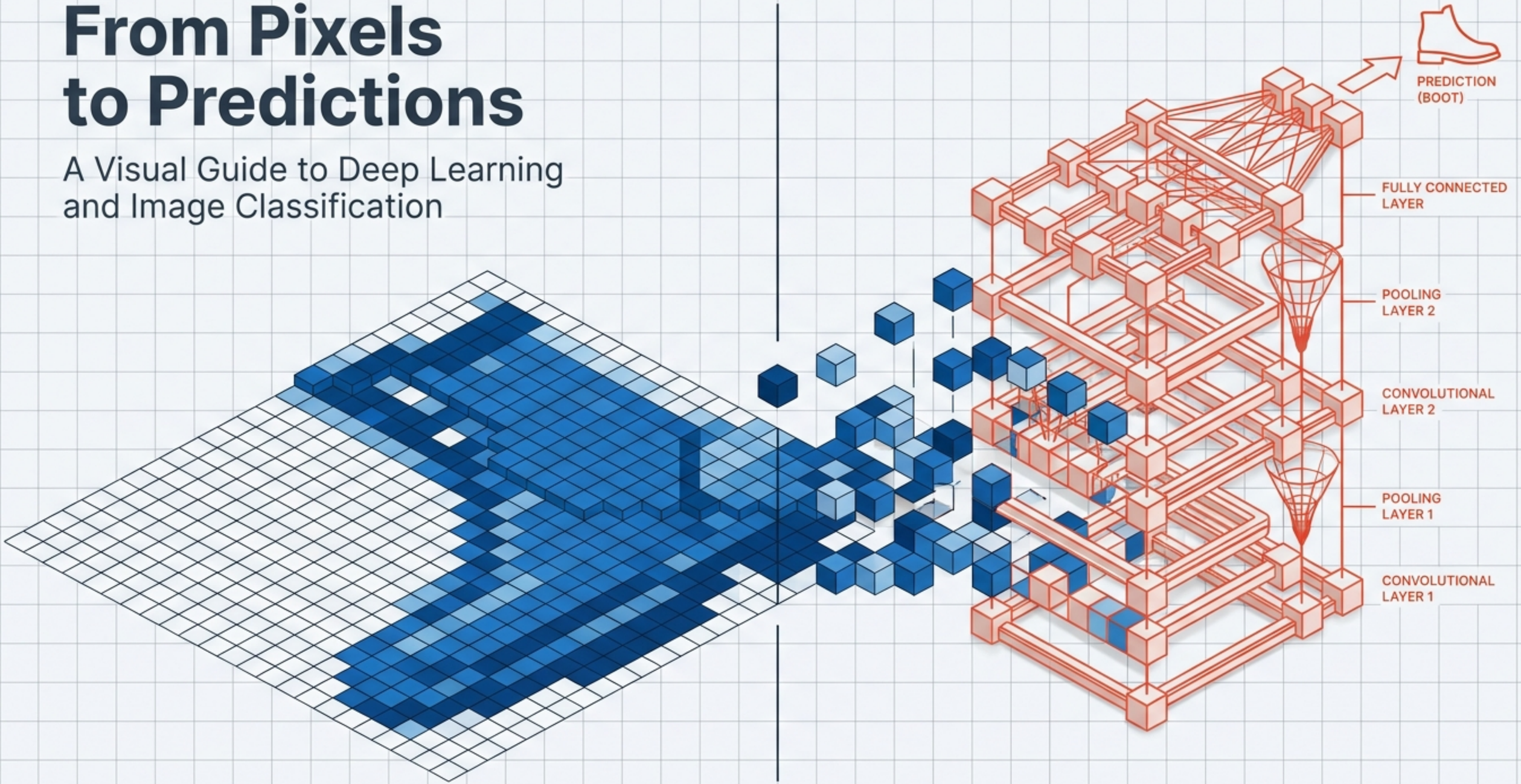
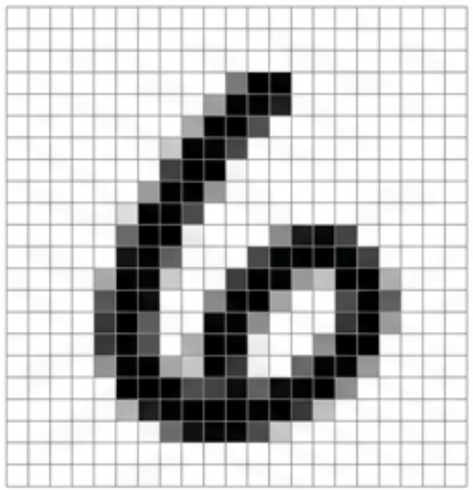
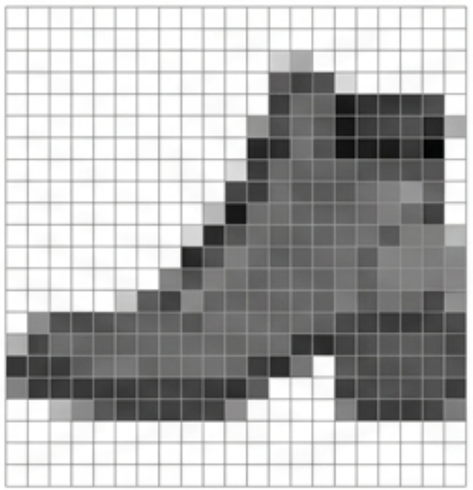
From Pixels to Predictions

A Visual Guide to Deep Learning and Image Classification



Gemini Notebook

The "Hello World" of Computer Vision

	Classic MNIST	Fashion MNIST
		
Subject	Handwritten Digits (0-9)	Clothing Items (T-shirts to Boots)
Total Images	70,000 (60k Train, 10k Test)	70,000 (60k Train, 10k Test)
Dimensions	28 × 28 Grayscale	28 × 28 Grayscale
Classes	10 distinct labels	10 distinct labels
Purpose	The historical starting point	A modern, slightly more challenging drop-in replacement

Gemini Notebook

Translating Reality into a Numeric Grid



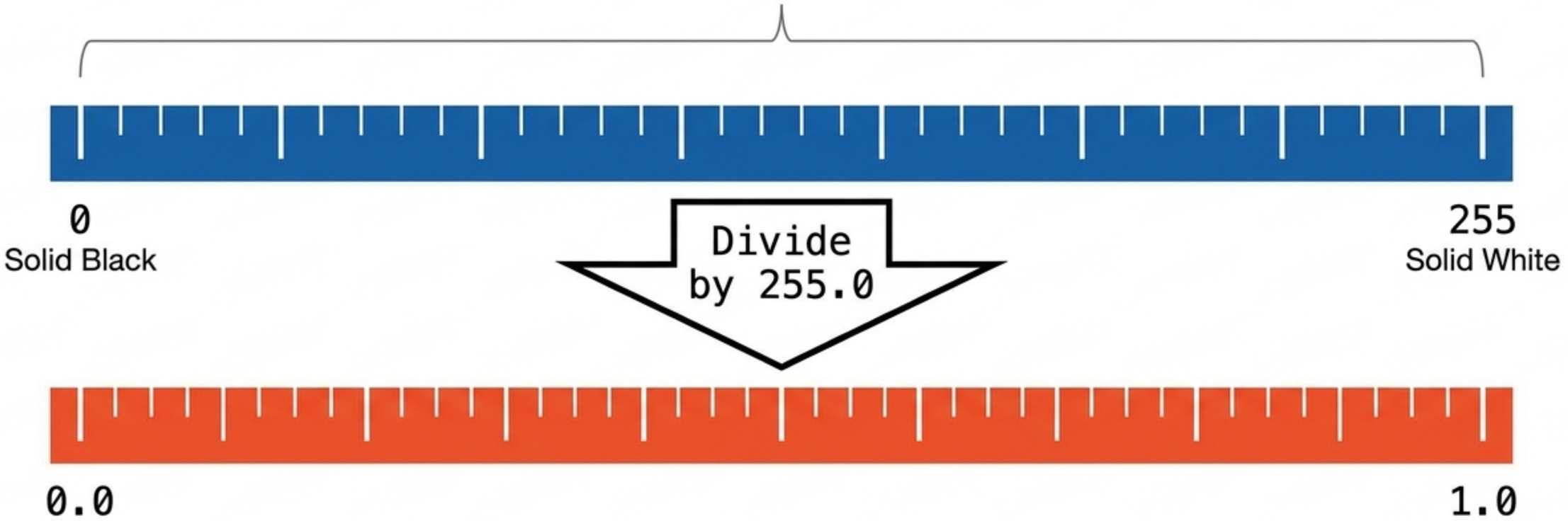
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	235	240	0	0	0	0	0
0	0	0	0	0	0	0	235	240	210	185	50	0	0
0	0	0	0	0	0	0	235	240	185	125	50	0	0
0	0	0	0	0	0	0	235	215	175	160	80	0	0
0	0	0	0	0	0	235	210	185	154	160	102	0	0
0	0	0	0	0	0	235	200	160	150	164	178	50	0
0	0	0	0	0	235	230	202	175	145	150	178	135	0
0	0	235	240	210	210	185	155	156	152	168	175	141	0
0	235	240	210	200	185	147	155	154	0	240	245	50	0
0	0	210	185	185	175	130	0	0	0	255	220	215	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

A computer does not see a “shoe.” It processes a mathematical matrix denoted as $X \in \mathbb{R}^{(28\times28)}$.

- **The Grid:** 28 rows × 28 columns = 784 total pixels.
- **The Intensity:** Each pixel holds an 8-bit brightness value.
  - 0 = Solid Black
  - 128 = Middle Gray
  - 255 = Solid White

Gemini Notebook

Preprocessing the Canvas



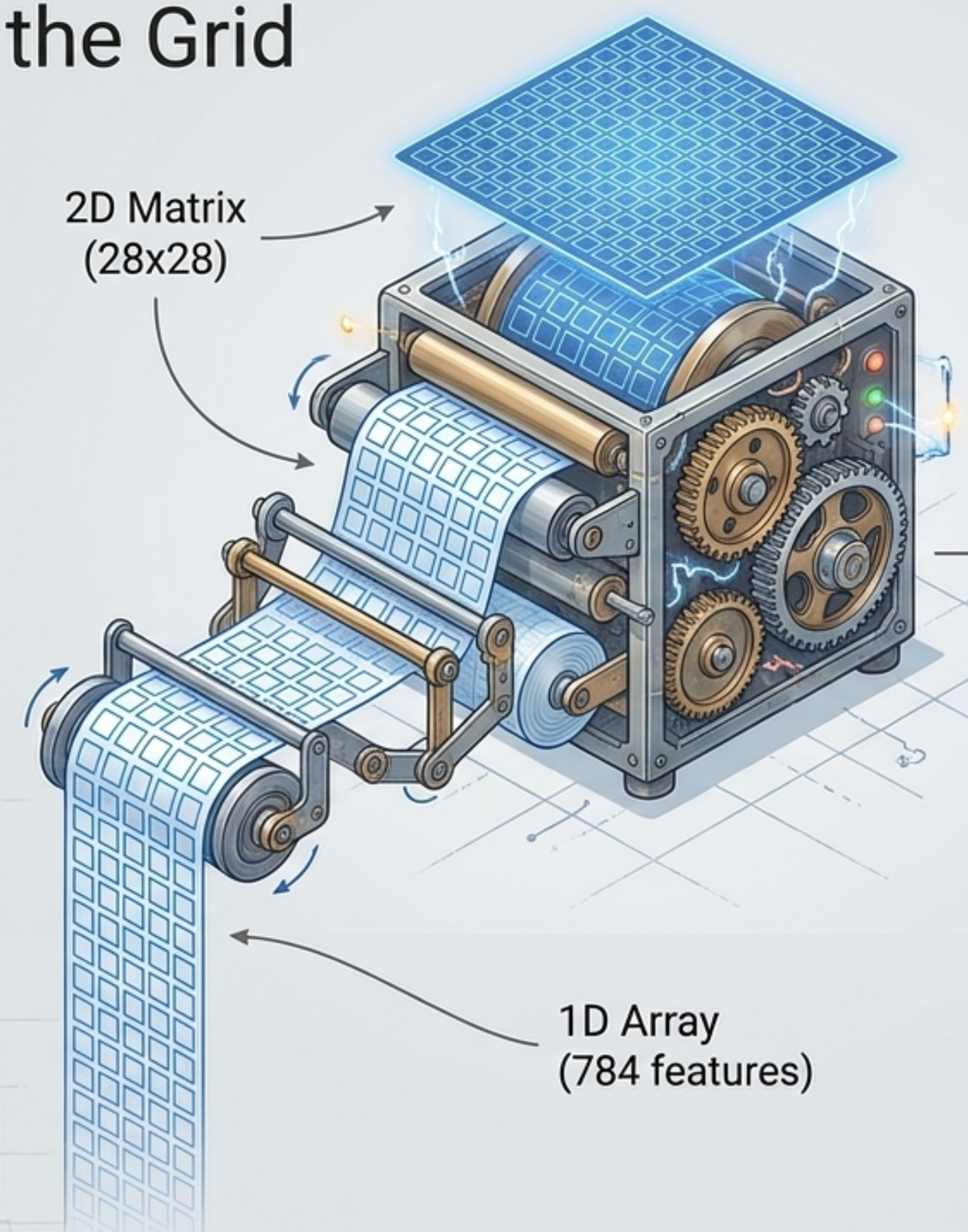

```
train_images = train_images / 255.0
```

The Golden Rule

Both the training dataset and the reserved testing dataset must be preprocessed identically. This ensures the model evaluates new data under the exact same conditions it learned from.

Gemini Notebook

Unspooling the Grid



Code

```
tf.keras.layers.Flatten(input
_shape=(28, 28))
```

The Problem

Standard Dense neural network layers cannot consume 2D matrices. They require a flat vector.

The Transformation

Converts the 28×28 matrix into a single 1D array of 784 features (named pixel0 through pixel783).

No learning happens in this step; it is purely a data formatting operation.

Gemini Notebook

Anatomy of the Neural Network

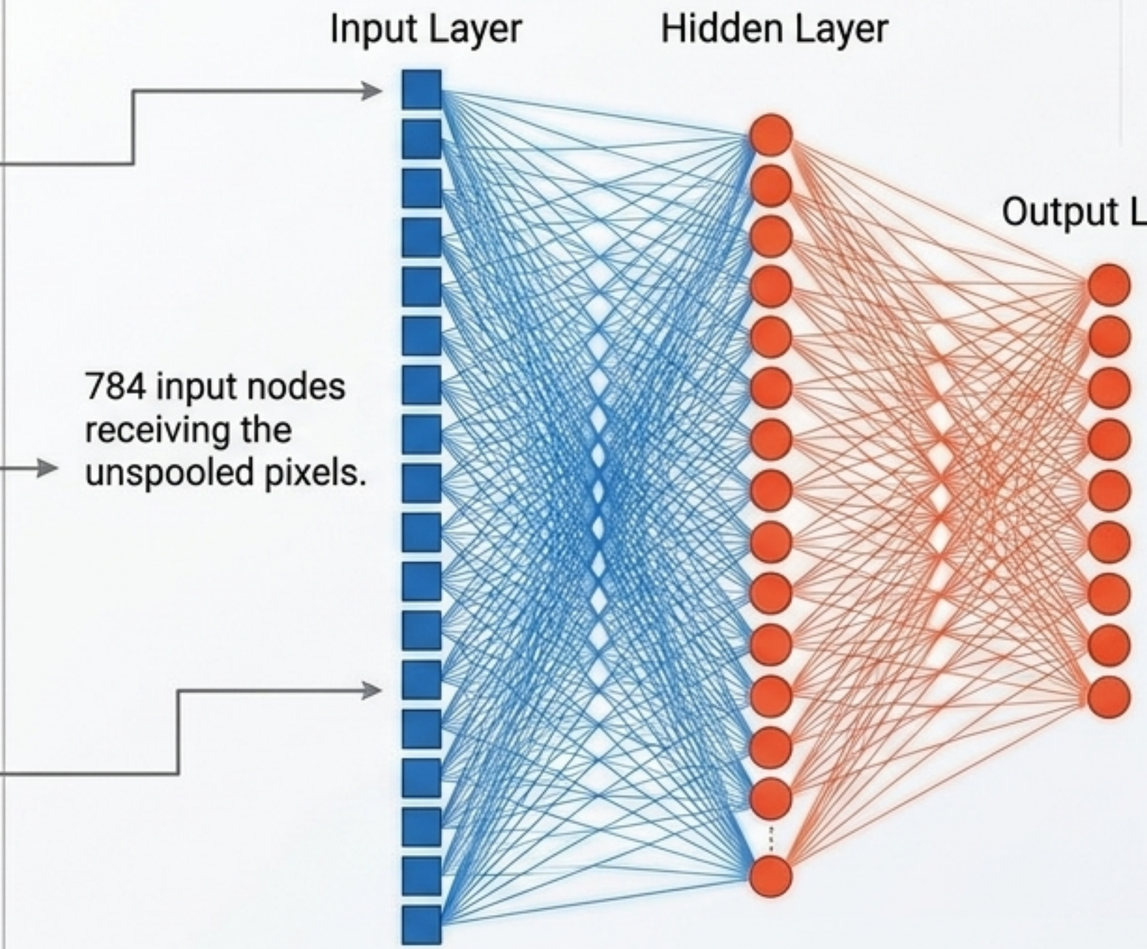
The Code

```
tf.keras.layers.Flatten(input
_shape=(28, 28))
```

```
tf.keras.layers.Dense(128,
activation='relu')
```

```
tf.keras.layers.Dense(10)
```

The Architecture



784 input nodes receiving the unspooled pixels.

10 final nodes.

Returns an array of "logits"—raw scores indicating how strongly the input matches the 10 clothing classes.

Gemini Notebook

Configuring the Learning Engine

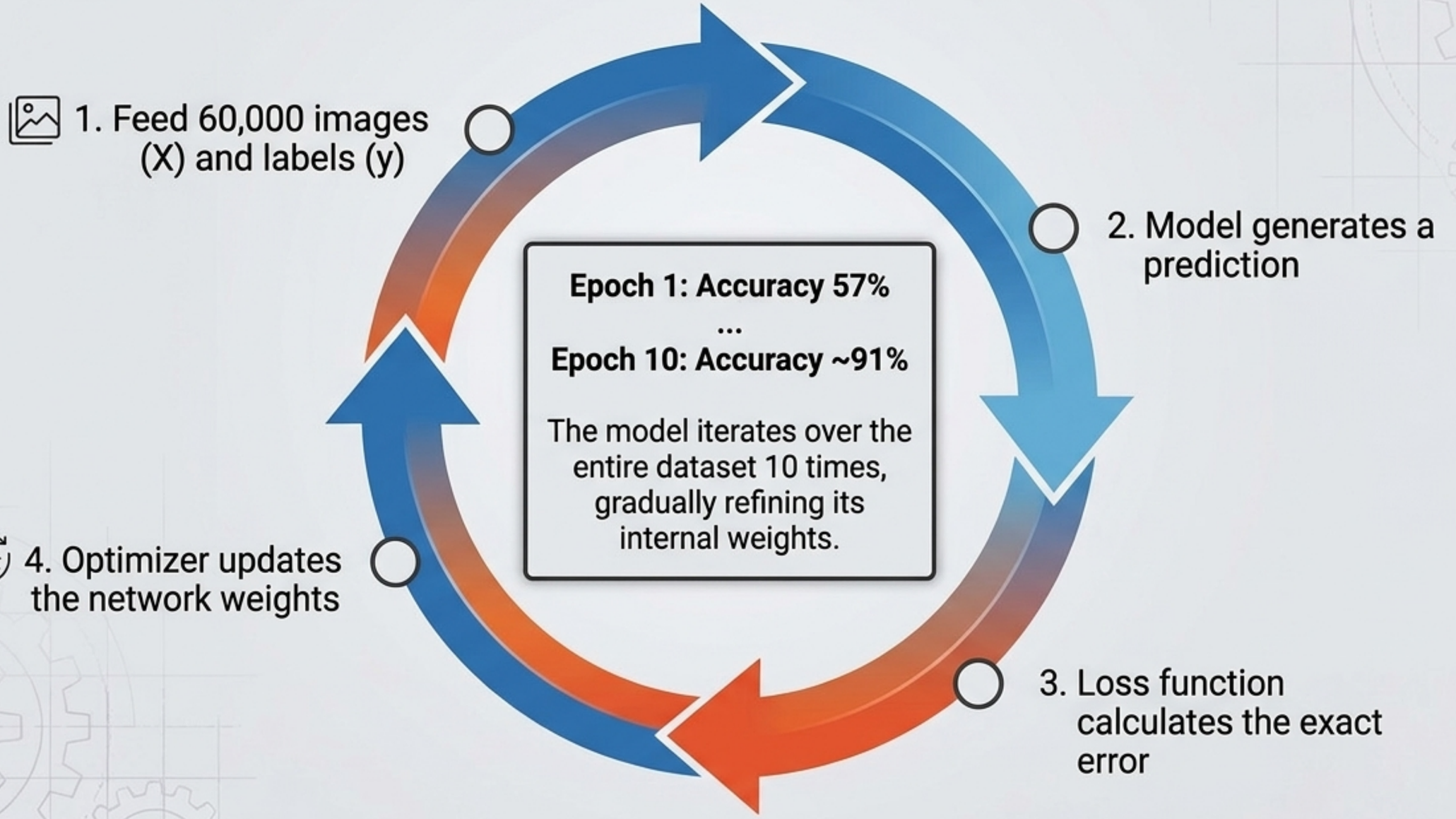
Before the network can learn, it needs rules for how to update its knowledge. This is the compile step.

		
Optimizer ('adam') The algorithm that dictates exactly how the model updates its internal weights based on the data it processes.	**Loss Function ('SparseCategoricalCrossentropy')** Measures how far the model's predictions are from the true labels. The engine's sole objective is to steer the model to minimize this loss.	**Metrics ('accuracy')** The human-readable dashboard gauge. It monitors the fraction of images correctly classified during the process.

Gemini Notebook

The Training Loop

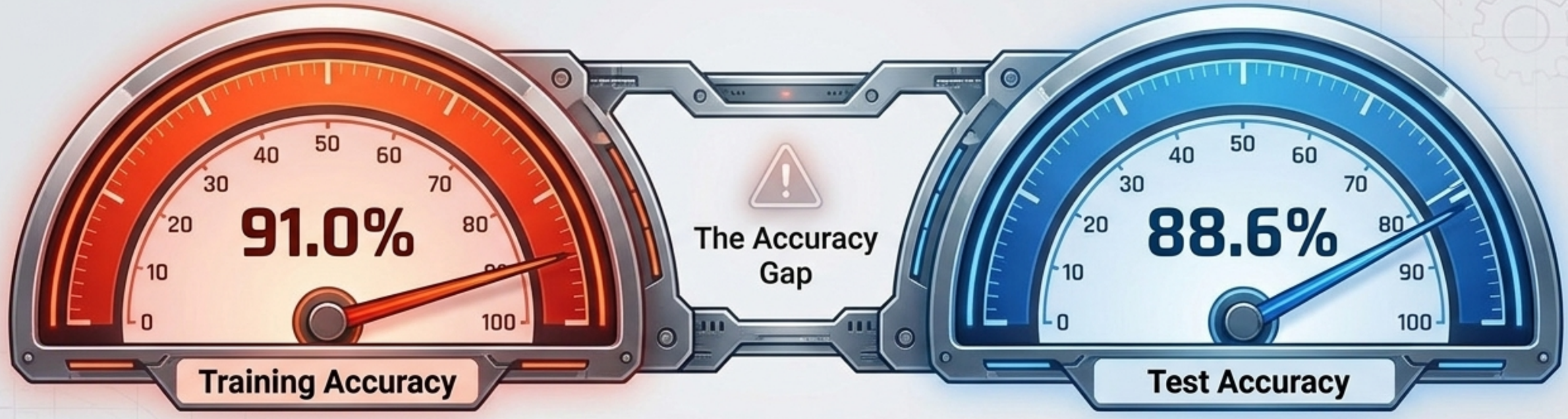
```
model.fit(train_images, train_labels, epochs=10)
```



Gemini Notebook

Testing Against the Unknown

To prove the model actually learned the concepts (and didn’t just memorize the answers), we evaluate it on 10,000 unseen test images.



The Diagnosis: Overfitting

The slight drop in performance on the test set is a classic symptom of overfitting. The model has ‘memorized’ the specific noise and quirks in the training data, rather than perfectly generalizing the core patterns of the clothing items.

Gemini Notebook

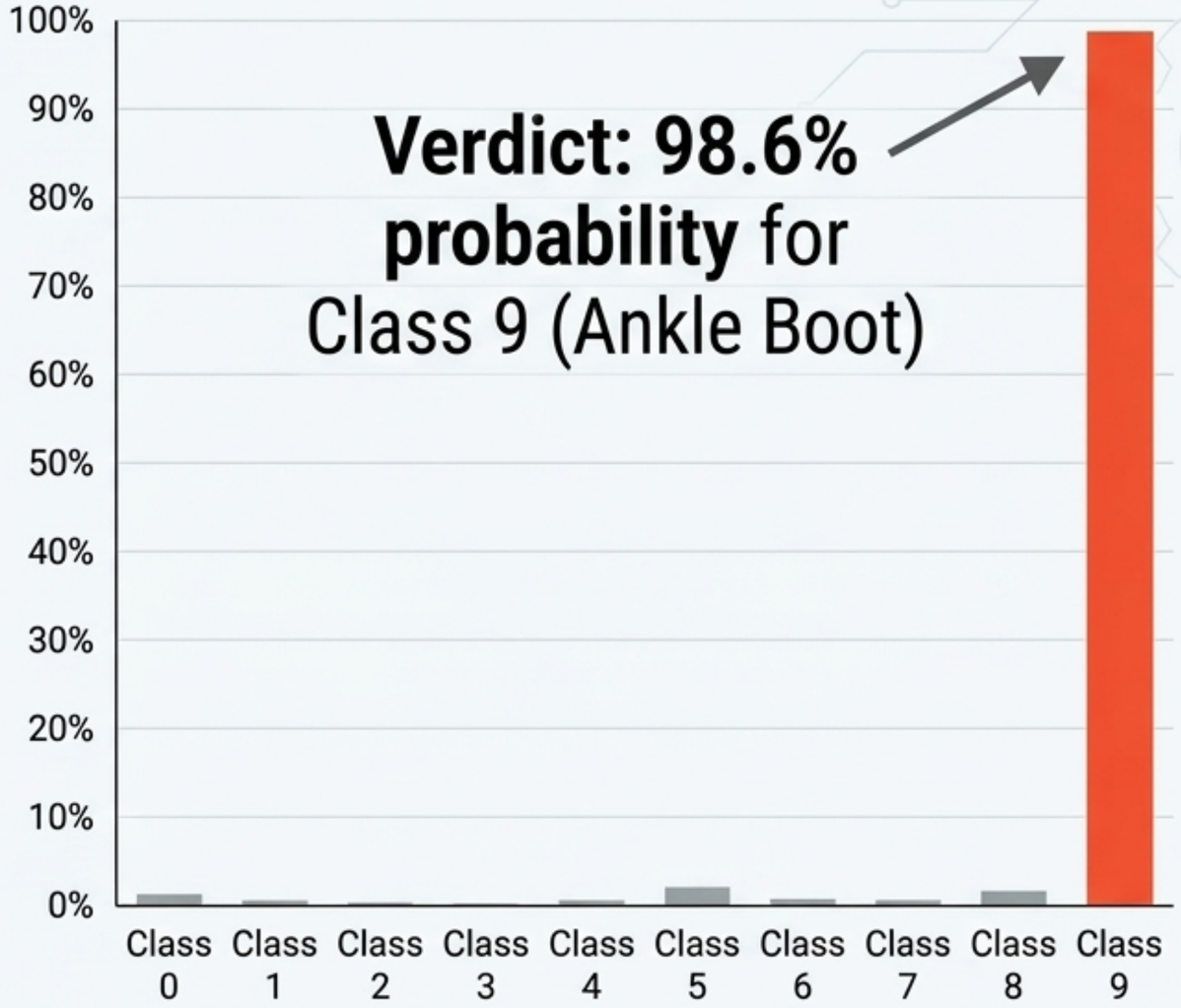
The Anatomy of a Prediction

To translate raw logits into human-readable probabilities, we attach a Softmax layer.

The model does not output a single word; it outputs an array of 10 confidence scores.

```
[6.8e-06, 8.7e-08, 2.3e-08, 1.7e-08,
9.2e-07, 1.5e-03, 3.3e-06, 1.1e-02,
4.7e-07, 9.8e-01]
```

The function **np.argmax()** automatically extracts the highest score.



Gemini Notebook

Trust, But Verify

Visual verification confirms real-world behavior. Crucial insight: A neural network can still output a highly confident score even when its prediction is entirely wrong.



T-shirt 100%



Trouser 98%



Bag 99%



Sneaker 100%



Sandal 75% (Actual: Sneaker)



Sandal 96%



Coat 97%



Dress 99%

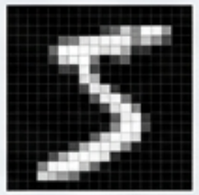


Pullover 95%

Gemini Notebook

The Universal Benchmarks

The Dataset Architecture:
28x28, 10 Classes



TensorFlow / Keras
Accessed via direct native integration.
`tf.keras.datasets.fashion_mnist`

Scikit-Learn
Fetched dynamically via the OpenML database.
`fetch_openml('mnist_784')`

PyTorch
Integrated directly into the examples repository.
`torchvision.datasets.FashionMNIST`

Takeaway: Because the dataset architecture is mathematically standardized, it serves as the ultimate, frictionless testing ground regardless of which framework a team chooses to deploy.

Gemini Notebook

The Complete Classification Pipeline

1. Ingest
Load 70,000 images and split into Train/Test arrays.

2. Preprocess
Divide pixels by 255.0 and Flatten the 2D matrix into a 784-element 1D vector.

3. Architecture
Chain simple representations (Dense 128 ReLU → Dense 10 logits).

4. Train & Compile
Steer the model with the Adam optimizer to minimize SparseCategoricalCrossentropy over 10 epochs.

5. Predict
Apply Softmax to convert raw logits into an array of 10 class probabilities, probabilities, and select the winner with argmax.

Gemini Notebook